

# Information Retrieval and Extraction — Assignment 1

Lexical and Semantic Retrieval on MIND and EB-NeRD  
Component 1: pipeline, candidate generation, evaluation harness

Noel Alex Jacob

Roll No. 2025201085

*CS4.406 Information Retrieval and Extraction — IIITH — Monsoon 2026*

**DRAFT — August 25, 2026.** Q1, Q2 and Q4 are built and measured; Q5 submission files are generated. **Q3 (semantic retrieval) is not yet built**, so every table below is lexical-versus-baselines only, and no leaderboard score exists yet. Sections marked [PENDING] are placeholders with the measurement named but not run. No number in this report is estimated: anything not measured is absent, not guessed.

## Contents

|                                                                             |   |
|-----------------------------------------------------------------------------|---|
| Summary of Deliverables                                                     | 2 |
| 1 What the assignment actually asks, in plain terms                         | 2 |
| 2 The two datasets, and the asymmetry that shapes every decision            | 3 |
| 3 Q1 — The pipeline                                                         | 3 |
| 3.1 The leakage boundary, and the honest limit of the test for it . . . . . | 3 |
| 3.2 The corpus decision, and the invisible ceiling it avoids . . . . .      | 4 |
| 4 Q2 — Lexical retrieval                                                    | 4 |
| 4.1 Do abstracts earn their place? . . . . .                                | 4 |
| 4.2 Parameter sweep — and what it says about tuning . . . . .               | 4 |
| 5 Q3 — Semantic retrieval [pending]                                         | 5 |
| 6 Q4 — The evaluation harness                                               | 5 |
| 6.1 Two regimes, kept strictly apart . . . . .                              | 5 |
| 6.2 Results — EB-NeRD, small tier . . . . .                                 | 6 |
| 6.3 Results — MIND, small tier . . . . .                                    | 6 |
| 6.4 The three uncomfortable observations . . . . .                          | 6 |
| 6.5 Beyond-accuracy — the trade-off, quantified . . . . .                   | 6 |
| 6.6 One metric legitimately has no confidence interval . . . . .            | 7 |
| 6.7 Slices . . . . .                                                        | 7 |
| 7 Q9 — Anti-gaming                                                          | 7 |
| 8 Q5 — Submissions                                                          | 7 |
| 9 Key decisions and rejected alternatives                                   | 8 |
| 10 Where this breaks at $10\times$ (Q6)                                     | 8 |
| 11 Limitations, stated plainly                                              | 9 |
| 12 Reproducing this report                                                  | 9 |

# Summary of Deliverables

Two news-recommendation datasets are built into one unified, temporally-split feature store; a lexical retriever and a full evaluation harness run over both. Every number in this report comes from a JSON artefact written by the run that produced it (`results/*.json`, `data/store/*/manifest.json`), not from a copied console log.

| Q  | Deliverable                                           | Where                                 | Status                          |
|----|-------------------------------------------------------|---------------------------------------|---------------------------------|
| Q1 | Reproducible pipeline, unified schema, temporal split | <code>src/data/</code>                | <b>done</b> — 42s rebuild       |
| Q2 | BM25 over title + abstract, recall@K                  | <code>src/retrieval/</code>           | <b>done</b> — 54-cell sweep     |
| Q3 | Embeddings + ANN retrieval                            | <code>src/retrieval/</code>           | NOT STARTED                     |
| Q4 | Metrics, beyond-accuracy, slices, bootstrap CIs       | <code>src/eval/</code>                | <b>done</b> — 72 tests          |
| Q5 | Codabench prediction files                            | <code>submissions/</code>             | files generated; <b>not sub</b> |
| Q6 | Design note ( $\leq 4$ pages)                         | —                                     | this draft is the source        |
| Q9 | Leakage test + serving-feature comparison             | <code>tests/test_no_leakage.py</code> | <b>done</b>                     |

**Code repository.** `github.com/N03L-22/IRE_A1_C1` — a personal repository, because the GitHub Classroom assignment had not been initialised at the time of writing. If Classroom is opened before the deadline the history transfers unchanged; the commit sequence is the same either way.

**One command rebuilds everything.** `makeclean&&makedata&&makestore&&makebm25&&makeeval`. The resource budget is an argument at every stage (`--n-jobs`, `--mem-gb`, capped at 28/28 all-inclusive) and the *resolved* values are written into every output file, so a number can always be traced back to the budget that produced it.

## 1 What the assignment actually asks, in plain terms

A user arrives at a news site. We are shown who they are, when they arrived, and a slate of articles that could be displayed. The task is to guess which they will click.

**The move that makes this a retrieval problem.** We do not know what this person wants, but we know what they read recently. So we paste together the headlines of their recent clicks and treat that text as a *search query*. “What will they click?” becomes “which articles match this query?”, and fifty years of search technique applies.

**Two ways to match, which fail differently.** *Lexical* matching counts shared words: they read something about an election, so find other articles containing “election”. Fast and explainable, but blind — “car” and “automobile” are unrelated strings. *Semantic* matching converts each article into a list of numbers capturing its meaning and finds articles whose numbers are close. It catches car/automobile but can drift into vaguely-related-but-wrong. Both are required precisely because they fail in different places.

**Why recall, and not accuracy.** This component is *candidate generation*: narrowing tens of thousands of articles to a few hundred worth ranking. The re-ranker is Component 2. So the only question that matters is *did the clicked article survive the cut* — which is what recall@K measures. Position 180 of 200 is a complete success here; being absent is fatal and unrecoverable downstream.

**The rule that can invalidate everything.** When evaluating on Tuesday, the model may only know what happened before Tuesday. If Wednesday’s clicks leak backwards, the scores look excellent and mean nothing. Hence a temporal split, never random, and a test that fails when the boundary breaks.

## 2 The two datasets, and the asymmetry that shapes every decision

|                          | MIND (English)     | EB-NeRD (Danish)            |
|--------------------------|--------------------|-----------------------------|
| Format                   | TSV, no header     | Parquet                     |
| Articles (small tier)    | 65,238             | 20,738                      |
| Labelled span            | <b>6.0 days</b>    | 14.0 days                   |
| Impressions, train split | 141,265            | 209,597                     |
| Median click history     | 19                 | 92                          |
| Article body text        | <b>absent</b>      | present                     |
| Click timestamps         | <b>absent</b>      | present                     |
| Publish time             | <b>absent (0%)</b> | present (100%)              |
| Session ids              | <b>absent (0%)</b> | present (100%)              |
| Provided embeddings      | <b>none</b>        | 300-d word2vec, 768-d mBERT |

**The governing constraint.** [F1, F10, F20, F29] Every richer feature EB-NeRD offers is one MIND lacks. Using any of them for a headline number silently converts a *dataset* comparison into a *feature-availability* comparison, and Q3.5 asks explicitly for cross-dataset observations. So:

**The shared retrieval path uses only the intersection — title, abstract, category, click history. Everything one-sided becomes a clearly-labelled single-dataset ablation.**

That single rule resolves four otherwise-independent decisions (encoder choice, recency weighting, body text, session context) and is the most load-bearing choice in the component.

### Consequences that surprised us.

- [F27] **A 7-day test split is impossible on MIND** — its entire labelled range is 6.0 days, so a 7-day test would consume the training data. The split *rule* is held constant and the realised spans differ: EB-NeRD 7 days, MIND 1 day.
- [F9] EB-NeRD’s *minimum* history length is 5, so a “cold start = fewer than 5 clicks” rule selects nobody there. Thresholds must be derived per dataset.
- [F29] EB-NeRD sessions average **1.9 impressions** — “session context” means roughly one prior impression. Measured, then dropped from C-1 (Sec. 9).

## 3 Q1 — The pipeline

Raw archives → per-dataset readers → unified schema → temporal split → parquet feature store. One reader per dataset knows its own file layout; nothing downstream of `src/data/readers.py` branches on which dataset it is looking at.

| Dataset | Train   | Val    | Test    | Realised span               |
|---------|---------|--------|---------|-----------------------------|
| MIND    | 141,265 | 15,700 | 73,152  | 2019-11-09 → 11-15 (6.0 d)  |
| EB-NeRD | 209,597 | 23,290 | 244,647 | 2023-05-18 → 06-01 (14.0 d) |

Realised proportions are 61/7/32 on MIND and 44/5/51 on EB-NeRD. [F4, F19] A literal 80/10/10 was rejected: it discards roughly 40% of EB-NeRD’s labelled impressions. We report the per-dataset proportions rather than claiming a uniform ratio.

### 3.1 The leakage boundary, and the honest limit of the test for it

For a test impression at time  $t$ , that user’s history is truncated to clicks strictly before  $t$ . `tests/test_no_leakage.py` asserts this on the built store *and* injects a deliberate post-boundary click to confirm the checker fails — a test that passes both before and after the boundary breaks would prove nothing.

**Stated limitation.** [F1] The invariant is verifiable only where per-click timestamps exist, i.e. **EB-NeRD**. MIND’s history is an untimestamped list, so on MIND the boundary rests on the authors’ construction and cannot be checked from the data. The store records this per row as `history_verifiable`, and the manifest as `history_boundary_verifiable:false`. Claiming the test covers MIND would be worse than the gap itself.

### 3.2 The corpus decision, and the invisible ceiling it avoids

[F2, F17, F20] MIND ships *two different news.tsv* files: 51,282 articles in train, 42,416 in dev, union 65,238. Dev therefore contributes **13,956 articles (21%) that train never saw**.

Measured over 61,028 sampled dev clicks: **23.3% land on articles absent from train’s file**. Indexing only the train corpus caps recall at **0.767** regardless of retriever quality — and that ceiling is invisible in the output, looking exactly like a weak model. We therefore index the **union** for every reported number, and run per-split as a separate ablation whose ceiling is stated.

This is not hypothetical: an early skeleton run capped the corpus arbitrarily and returned recall 0.0000 everywhere, because 372 of 379 clicked articles fell outside the kept slice.

## 4 Q2 — Lexical retrieval

BM25 over **title + abstract** (Q2.1), **bm25s** backend, with an independent ~50-line textbook implementation used as a correctness check. Tokenisation is regex word tokens, lowercased, NFC-normalised, no stemming and no stopword removal — applied identically to both languages.

**Why no stemming.** An English stemmer on Danish is actively harmful, and using a *different* stemmer per dataset would confound the cross-dataset comparison. IDF already down-weights common terms, making stopword removal largely redundant. Per-language stemming is named as the obvious ablation.

**Why NFC is not optional.** Danish å can be encoded as one code point or as base+combining pair; two encodings of one word are two index terms, silently halving recall on affected queries. Applied to corpus *and* query, and asserted in a test. Note that ø and æ do *not* decompose — our first test fixture used ø and was therefore testing nothing.

### 4.1 Do abstracts earn their place?

Q2.1 mandates title + abstract. We measured what that actually contributes.

| Corpus  | Title tokens | Abstract tokens | Abstract share | Empty abstracts |
|---------|--------------|-----------------|----------------|-----------------|
| MIND    | 11.2         | 36.1            | <b>76.3%</b>   | 3,415 (5.2%)    |
| EB-NeRD | 6.8          | 18.3            | <b>72.8%</b>   | 1,709 (8.2%)    |

Abstracts are roughly three times the title and **dominate the index**. Their measured contribution (EB-NeRD, BM25 + 24 h window,  $n = 800$ ):

| Indexed text     | recall@50 |
|------------------|-----------|
| Title + abstract | 0.2475    |
| Title only       | 0.2362    |

[F28] +0.011, against a CI half-width of  $\pm 0.030$  — **statistically indistinguishable**. Three quarters of the index buys an effect too small to detect at this sample size. We keep the field pair because Q2.1 is binding, and report the null result as a null result: *the mandated field pair is not measurably better than titles alone*. The MIND title-only arm timed out and is still to be run; no MIND figure is quoted.

### 4.2 Parameter sweep — and what it says about tuning

Full grid  $k_1 \in \{0.9, 1.2, 1.6\} \times b \in \{0.3, 0.75, 1.0\} \times \text{last\_n} \in \{5, 15, 50\} \times \text{window} \in \{24\text{h}, \text{none}\}$ , run **on val only**, 54 cells in 44s across 12 workers.

| Factor                               | Range (EB-NeRD, recall@100)                                | Verdict                           |
|--------------------------------------|------------------------------------------------------------|-----------------------------------|
| <b>Recency window</b> (24 h vs none) | 0.0075 $\rightarrow$ 0.2475 — <b>33<math>\times</math></b> | dominates everything              |
| <b>last_n</b> (5/15/50)              | 0.2162 $\rightarrow$ 0.2475                                | modest; 15 peaks, 50 worst        |
| <b>b</b> (0.3/0.75/1.0)              | 0.2313 $\rightarrow$ 0.2475                                | small, in the predicted direction |
| <b>k<sub>1</sub></b> (0.9/1.2/1.6)   | 0.2362 $\rightarrow$ 0.2475                                | <b>smallest</b>                   |

[F23] The ordering *is* the finding: **tuning BM25 is nearly pointless here compared with deciding what pool it searches**.  $k_1$  was predicted to barely matter (titles rarely repeat a term) and did not. Parameters chosen on val before test was touched: EB-NeRD  $k_1=1.6, b=1.0, n=15$ ; MIND  $k_1=1.6, b=0.75, n=5$ .

**Do not over-claim these margins.** The top-to-bottom spread within the 24 h arm is 0.2162–0.2475 at  $n = 800$ , comfortably inside the CI width. The parameter choice is defensible *as a procedure*, not as a finding.

## 5 Q3 — Semantic retrieval [pending]

Not yet built. The decisions are settled and recorded; the measurements are not taken, so no numbers appear here.

**Planned:** own embeddings from `xlm-roberta-base` (768-d) as primary — the brief names XLM-RoBERTa explicitly, and it runs on both datasets. Ablations: `xlm-roberta-large` (1024-d, the capacity question and the Q6 scale anchor) and a similarity-trained `paraphrase-multilingual-MiniLM-L12-v2` (384-d). EB-NeRD’s provided 768-d mBERT vectors are a same-dimension baseline; its 300-d word2vec a weaker one.

**The risk being controlled for.** XLM-R and BERT are trained for masked-language modelling, *not* to make cosine similarity meaningful. Their [CLS] vectors are known to occupy a narrow cone where everything looks similar. The failure is silent — plausible but mediocre numbers, easily misdiagnosed as “semantic retrieval does not work here”. Mitigations: mean-pool the final hidden states (never [CLS]), L2-normalise, and keep the similarity-trained MiniLM row so the hypothesis is *measured* rather than assumed. A Danish sanity probe — 20 known-related pairs against 20 unrelated — gates all of it.

**ANN index.** [F31] Brute force on demo/small (Q3.2 permits it, and it is *exact*, giving the recall ceiling ANN is measured against); FAISS HNSW at large scale. Benchmarked with random vectors, 28 threads:

| Corpus                | Exact    | HNSW $ef=128$ | Speed-up | recall vs exact |
|-----------------------|----------|---------------|----------|-----------------|
| EB-NeRD small, 768-d  | 190 ms   | 14.8 ms       | 12.8×    | 0.7700          |
| MIND small, 768-d     | 888 ms   | 36.3 ms       | 24.5×    | 0.5778          |
| EB-NeRD large, 1024-d | 1,677 ms | 60.1 ms       | 27.9×    | 0.4461          |

**These are a worst case and must be re-run on real vectors.** Uniform-random vectors in 768–1024 dimensions are near-orthogonal, the hardest possible case for a proximity graph. Real embeddings cluster, so measured recall should be substantially higher. What the benchmark *does* establish regardless of distribution: the latency, the build times (4.1 s / 23.3 s / 74.4 s), the memory (514 MB at large), and the shape of the  $ef$  trade-off. **ANN recall must never be reported without the exact baseline beside it** — a low  $ef$  silently caps recall and looks like a weak encoder.

## 6 Q4 — The evaluation harness

One function, `evaluate(retriever, split, ...)`, scores every retriever. Q4.5 is satisfied by construction: there is nowhere else a metric can be computed. Output is a tidy table — one row per dataset × retriever × slice × metric × feature-set — so the tables below are a group-by, not a copy-paste.

### 6.1 Two regimes, kept strictly apart

|            | Corpus regime                         | Slate regime                            |
|------------|---------------------------------------|-----------------------------------------|
| Candidates | every article (21K–65K)               | the ~11 shown in that impression        |
| Question   | did the click survive the cut?        | did you rank it above the others shown? |
| Metric     | recall@K                              | AUC, MRR, nDCG                          |
| Who cares  | candidate generation (this component) | the leaderboard                         |

Conflating them produces numbers that look fine and mean nothing — computing AUC over the whole corpus makes almost everything a negative and yields a spectacular, meaningless score. Every result row carries a **regime** column.

## 6.2 Results — EB-NeRD, small tier

$n = 800$  evaluated impressions,  $B = 1000$  seeded bootstrap, 95% percentile CIs.

| Retriever                         | recall@50                      | recall@100                     | recall@200                     |
|-----------------------------------|--------------------------------|--------------------------------|--------------------------------|
| <b>Recency</b> (ignores the user) | <b>0.9050</b> [0.8850, 0.9237] | <b>0.9463</b> [0.9300, 0.9600] | <b>0.9688</b> [0.9562, 0.9800] |
| BM25 + 24 h window                | 0.2475 [0.2175, 0.2775]        | 0.2475 [0.2175, 0.2775]        | 0.2475 [0.2175, 0.2775]        |
| BM25, full corpus                 | 0.0075 [0.0025, 0.0138]        | 0.0125 [0.0050, 0.0200]        | 0.0200 [0.0100, 0.0300]        |
| Popularity                        | 0.0063 [0.0013, 0.0125]        | 0.0100 [0.0037, 0.0163]        | 0.0262 [0.0150, 0.0375]        |
| Random                            | 0.0000                         | 0.0000                         | 0.0000                         |

## 6.3 Results — MIND, small tier

| Retriever         | recall@50                      | recall@100                     | recall@200                     |
|-------------------|--------------------------------|--------------------------------|--------------------------------|
| <b>Popularity</b> | <b>0.0468</b> [0.0340, 0.0609] | <b>0.0682</b> [0.0531, 0.0861] | <b>0.1415</b> [0.1187, 0.1649] |
| BM25              | 0.0116 [0.0054, 0.0189]        | 0.0159 [0.0086, 0.0240]        | 0.0217 [0.0136, 0.0310]        |
| Random            | 0.0004 [0.0000, 0.0013]        | 0.0007 [0.0000, 0.0017]        | 0.0007 [0.0000, 0.0017]        |

## 6.4 The three uncomfortable observations

**1. Recency dominates everything.** [F16, F21] 94.3% of EB-NeRD clicks are on articles under 24 hours old, while only  $\sim 1.1\%$  of the corpus is that fresh. A retriever that ignores the user entirely and returns the newest articles achieves  $\text{recall@50} = 0.9050$ . **Candidate generation on this dataset is primarily a recency-filtering problem and only secondarily a text-matching one.** The interesting question shifts from “lexical or semantic?” to “given the  $\sim 132$  fresh articles, which does the user pick?” — which is exactly what a re-ranker answers.

This is legitimate, not a leak: publish time is known at serving time. Ranking by *future* engagement would be a leak; filtering on `published_time < t` is not.

**2. BM25 is statistically indistinguishable from popularity on EB-NeRD, and loses to it on MIND.** [F21, F25] The full-corpus BM25 and popularity CIs overlap on EB-NeRD. On MIND, popularity beats BM25 with non-overlapping CIs at every K — and MIND has no publish times, so the 24-hour window that rescues BM25 on EB-NeRD is **unavailable there**. The cross-dataset asymmetry is not a detail; it is the result.

Replacing the skeleton’s crude token-overlap scorer with real BM25 changed almost nothing on the full corpus ( $0.0000 \rightarrow 0.0075$ ). The scoring function was never the limiting factor — the retrieval pool was.

**3. The slate metrics barely discriminate.** [F25]

| EB-NeRD       | AUC                            | nDCG@10                        |
|---------------|--------------------------------|--------------------------------|
| BM25 + 24 h   | 0.5243 [0.5002, 0.5467]        | 0.4657 [0.4465, 0.4861]        |
| Recency       | 0.5047 [0.4841, 0.5240]        | 0.4281 [0.4095, 0.4487]        |
| <b>Random</b> | <b>0.4810</b> [0.4591, 0.5022] | <b>0.4191</b> [0.4019, 0.4373] |

Every AUC is within noise of 0.5, and random ranking scores  $\text{nDCG@10} = 0.42$ . Not a bug: slates average 11–12 items with exactly one click, so  $\text{nDCG@10}$  covers nearly the whole slate and a random permutation places the single positive respectably often. **Any slate-regime table without a random baseline row is misleading** — 0.466 looks decent until the floor is known to be 0.419.

## 6.5 Beyond-accuracy — the trade-off, quantified

| EB-NeRD retriever | Diversity                  | Novelty (bits) | Coverage              |
|-------------------|----------------------------|----------------|-----------------------|
| BM25, full corpus | 0.6979 [0.6907, 0.7046]    | 12.61          | <b>0.5037</b> (no CI) |
| BM25 + 24 h       | 0.8080 [0.8056, 0.8104]    | 11.67          | 0.0110 (no CI)        |
| Recency           | 0.8313 [0.8308, 0.8319]    | 11.30          | 0.0147 (no CI)        |
| Popularity        | 0.7924 (zero width)        | <b>8.64</b>    | 0.0096 (no CI)        |
| Random            | <b>0.8650</b> (zero width) | 12.71          | 0.0096 (no CI)        |

[F26] Three things worth stating:

- **The recency filter costs 46× coverage** ( $0.5037 \rightarrow 0.0110$ ) while buying 33× recall. That is the sharpest accuracy-versus-coverage trade-off measured.
- **Popularity’s novelty is 4 bits below everything else** — it recommends exactly what everyone already clicked. Its zero-width CI is correct, not a bug: it returns the same list to every user, so there is no between-impression variation to bootstrap.
- **Random has the highest diversity.** Diversity alone is not a quality signal; it must always be read against an accuracy column.

## 6.6 One metric legitimately has no confidence interval

**[F24]** D5 requires a bootstrap CI on every metric. **Coverage cannot honestly have one**, and the failure was caught by the harness’s own output: BM25’s coverage came out as 0.3933 [0.3432, 0.3671] — **the point estimate outside its own interval**, impossible for a valid percentile bootstrap.

Coverage counts *distinct* articles, so it grows monotonically with the number of impressions evaluated. Resampling  $n$  units with replacement makes  $\sim 37\%$  duplicates, so every resample sees fewer unique articles than the real sample and the whole distribution sits below the estimate.

| Scheme                            | Result (full-sample coverage 0.9790)                    |
|-----------------------------------|---------------------------------------------------------|
| Resample $n$ with replacement     | 0.9783 vs [0.9035, 0.9235] — excludes the point         |
| Subsample $m/n = 0.50$            | mean 0.8532 — still excludes                            |
| $m/n = 0.80 / 0.90 / 0.95 / 0.99$ | 0.9540 / 0.9688 / 0.9745 / 0.9782 — all exclude         |
| $m = n$ without replacement       | the original sample: zero variance, zero-width interval |

No ratio works; the bias vanishes only as  $m \rightarrow n$ , where no resampling remains. Coverage is therefore reported as a point estimate with explicit NaN bounds and a (no CI) marker. **Saying so is a stronger answer than shipping a plausible-looking interval that is biased by construction.** Coverage remains comparable across retrievers on the same impression sample, which is how it is reported.

## 6.7 Slices

Thresholds are derived per dataset from the measured distribution, never fixed constants, and are reported with every sliced number. A boundary chosen after seeing which one gives a nicer result is not a finding. Cold start is defined as fewer than 7 clicks; EB-NeRD’s minimum history is 5, so this selects a narrow band there and the realised slice size is reported alongside.

## 7 Q9 — Anti-gaming

**Metrics with and without serving-unavailable features.** A live recommender ranking at 09:00 cannot know how long a user will read an article it has not yet shown them. EB-NeRD’s training rows carry `read_time`, `scroll_percentage`, `next_read_time` and `next_scroll_percentage` — all recorded *after* the click. Predicting a click from its own consequences is circular. **[F6]**

The organisers settled the boundary for us: **those four columns are exactly what is deleted from the test set.** They are excluded in `clean.py` with a comment naming why, so a future feature-store change cannot quietly reintroduce them.

Our retrievers never touch those fields, so the naive with/without gap is zero. The honest pair we do have is the **cold-start popularity fallback**, since popularity computed over the evaluation window is future knowledge. Every fallback result carries an `is_fallback` flag, so the comparison is a filter on the results table rather than a second run.

**The behaviour-window test.** Described in Sec. 3.1, including the honest statement that it covers EB-NeRD only.

## 8 Q5 — Submissions

Both leaderboards take the same format: one line per impression, `impression_id[rank1,...,rankN]`, a permutation aligned to the candidate list *in the order given*. It is a mark-up of the slate handed to you, not a re-ordering — emitting a preferred order is scored as a different answer entirely.



**A 16× algorithmic fix was required.** [F32] The first run produced predictions at **162 impressions/s** — about 4 hours per file — because it ran a top-500 retrieval over all 120,961 articles per impression, then discarded everything outside that impression’s ~37-item slate.

| Approach                                          | Rate           | Verdict                                 |
|---------------------------------------------------|----------------|-----------------------------------------|
| <code>retrieve(k=500)</code> over the full corpus | 162/s          | 4 hours per file                        |
| <code>bm25s weight_mask</code> on the slate       | 173/s          | exact, but masks selection not the scan |
| <b>Doc-major scoring of the slate only</b>        | <b>2,570/s</b> | <b>16×; ~15 min</b>                     |

The fast path is not numerically identical, and that is deliberate. `retrieve()` delegates to `bm25s` (Lucene IDF variant); `score_subset()` uses the textbook Robertson formula. Reproducing `bm25s`’s exact scores was attempted and abandoned — a careful replication of its documented formula still disagreed by up to 106 in absolute score on the real corpus. What was verified instead is that **the two produce equivalent rankings**: 0 discordant pairs out of 780. A regression test asserts zero discordant pairs rather than list equality, because a score *tie* broken differently is not a disagreement. This is acceptable **only** because the submission format is a permutation — `score_subset()` must never be used for a reported metric.

**Status.** MIND prediction file generating at the time of writing (2,370,727 impressions); EB-NeRD test data staged (13,536,710 rows, 13,336,711 unique impression ids — **~200K ids repeat, so a naive row-wise write emits duplicate lines**). **Neither has been submitted, and no leaderboard score exists.** Screenshots (Q7.3) are therefore pending.

**What is currently being submitted is a weak baseline, and should be read as one.** The retriever is BM25 at val-chosen parameters, which Sec. 6.4 shows is indistinguishable from popularity. The submission exists to de-risk the format and the memory path early; it is not a result.

## 9 Key decisions and rejected alternatives

The full record with costs is in `decisions.md`. The ten that matter:

| Decision        | Chosen                                                 | Rejected, and why                                                                           |
|-----------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Unified schema  | Intersection only: title, abstract, category, history  | Richer per-dataset fields — would turn a dataset comparison into a feature-availability one |
| Split rule      | Hold out official test, carve val from train tail      | Literal 7 days (impossible on MIND); literal 80/10/10 (discards 40% of EB-NeRD)             |
| MIND corpus     | Union (65,238) for headline, per-split as ablation     | Per-split as headline — invisible 0.767 recall ceiling                                      |
| Lexical fields  | Title + abstract (mandated), title-only reported       | Title+body — MIND has no body, breaking comparability                                       |
| Query           | Last 20 clicks, logarithmic position decay             | Exponential decay (collapses to last 2–3 titles); plain mean (bland over 160 clicks)        |
| Cold start      | < 7 clicks; rising + popular fallback, train-only      | Fixed < 5 (selects nobody on EB-NeRD); popularity alone (stale in news)                     |
| Encoder         | Own <code>xlm-roberta-base</code> 768-d, both datasets | Provided vectors as primary — EB-NeRD only, so MIND would use a different model             |
| ANN             | Brute force on small (exact), FAISS HNSW on large      | ScaNN (TensorFlow-coupled); ANN-only (no exact ceiling to measure against)                  |
| Session context | <b>Dropped from C-1</b> , deferred to C-2              | Building it — 0% coverage on MIND, and 1.9 impressions/session on EB-NeRD                   |
| Re-ranker       | Interface defined, identity implementation only        | Building a real ranker — that is C-2, and costs C-1’s marks                                 |

## 10 Where this breaks at 10× (Q6)

- [F15] **The join breaks before the model does.** EB-NeRD large is 52× the impressions of small and its history parquet is larger than its behaviours file. The data layer, not the retriever, is the first thing to give.
- [F31] **ANN recall degrades with scale.** Against exact search, HNSW at  $ef=128$  falls  $0.77 \rightarrow 0.45$  as the corpus grows 6× and the dimension 1.33×. Index build goes  $4.1\text{ s} \rightarrow 74.4\text{ s}$ ; vectors  $64\text{ MB} \rightarrow 514\text{ MB}$ .



- **[F32] The prediction path had to change algorithm, not just scale.** Full-corpus retrieval per impression was 4 hours per file; scoring the slate directly is  $\sim 15$  minutes. This is the concrete  $10\times$  lesson: the naive implementation was adequate at small and unusable at large.
- **Not yet measured:** encoder throughput on 86K articles, and the real-vector HNSW recall that would replace the pessimistic random-vector figures.

## 11 Limitations, stated plainly

1. **Q3 is not built.** No semantic retriever, so no lexical-versus-semantic comparison — the substance of Q3.5 — exists yet.
2. **No leaderboard scores.** Prediction files are generated but not submitted.
3.  $n = 800$  **per dataset** for the CI-bearing results. CI half-widths of  $\pm 0.03$  mean several reported differences are not statistically separable, and the report says so at each point rather than once.
4. **MIND’s leakage boundary is unverifiable from data** and rests on the authors’ construction.
5. **HNSW figures use random vectors** and are a pessimistic bound; they must be re-run on real embeddings.
6. **MIND’s title-only ablation timed out** and is not reported.
7. **MIND’s test window is a single day**, so one day’s topical anomaly moves every MIND number.

## 12 Reproducing this report

```
make clean && make data && make store # Q1: raw -> unified, temporally split store
make bm25 # Q2: BM25 vs baselines, both datasets
make bm25-sweep # Q2: 54-cell parameter sweep, val only
make eval # Q4: full harness, bootstrap CIs
make test # 72 tests, incl. the leakage mutation test
python -m src.submit.codabench --dataset mind --tier large # Q5
```

Artefacts: `results/eval_*.json` (every result row with CI,  $n$ , regime and slice basis), `results/bm25_sweep_*.json`, `data/store/*/manifest.json` (split spans, counts, resolved budget), `submissions/*.meta.json`. The findings cited as **[Fnn]** are in `plan/execution_plan_log.md`.